Semantiche di esecuzione e architetture parallele per una formulazione continua della Particle Swarm Optimization

Giovanni Giuseppe Iacuzzo[1,*]

[1]Dipartimento di Ingegneria e Architettura, Università degli Studi di Enna "Kore", Enna, Italia

[*]Autore corrispondente: giovanni.iacuzzo@unikore.it

Sommario

La parallelizzazione della Particle Swarm Optimization (PSO) non è una scelta puramente implementativa: a seconda del punto in cui si introducono sincronizzazione, pubblicazione degli aggiornamenti e migrazione, essa può preservare la traiettoria del metodo seriale oppure definire una dinamica di ricerca differente. Questo lavoro studia cinque modalità di esecuzione di una formulazione continua della PSO (CPSO), implementate in un motore CPU a memoria condivisa: seriale sincrona, master–slave sincrona, particle-parallel sincrona, asincrona event-driven e a isole. Per le due modalità sincrone parallele il contratto di aggiornamento è costruito per preservare la traiettoria seriale; per le modalità asincrona e a isole la variazione della dinamica è oggetto esplicito dell'analisi. Il software implementa sette topologie di comunicazione tra particelle: global-best, ring, von Neumann, wheel, random connessa, small-world e scale-free. La campagna architetturale principale studia le tre topologie canoniche global-best, ring e von Neumann; una campagna complementare estende il confronto a tutte e sette. Il protocollo interno utilizza sei funzioni analitiche deterministiche in dimensione 30 e 100, mentre la misurazione dello scaling è separata e usa un carico sintetico controllabile. Un livello esterno confronta la CPSO con dieci baseline PSO discrete in spazio continuo su COCO BBOB, BBOB-LargeScale e CEC 2013 LSGO, a budget di valutazioni della funzione obiettivo (FE) abbinato. Il manoscritto specifica problema, architetture, motivazioni bibliografiche e protocolli riproducibili; i risultati confirmatory sono deliberatamente lasciati da inserire dopo l'esecuzione delle campagne e non vengono anticipati.

Parole chiave: Particle Swarm Optimization; PSO a tempo continuo; parallelizzazione; algoritmi sincroni e asincroni; modelli a isole; equivalenza algoritmica; analisi sperimentale.

1 Introduzione

La Particle Swarm Optimization (PSO) è una metaeuristica di popolazione nella quale ciascun agente aggiorna il proprio stato combinando memoria individuale e informazione condivisa con il vicinato [1]. La parallelizzazione è naturale quando le valutazioni della funzione obiettivo sono indipendenti, ma non è neutrale rispetto all'algoritmo: una barriera di generazione, una pubblicazione immediata dei migliori personali o una migrazione tra sottopopolazioni determinano quali informazioni siano disponibili a ciascuna particella e in quale istante. Di conseguenza, due implementazioni che eseguono lo stesso numero di valutazioni possono seguire traiettorie diverse e non devono essere confrontate come se differissero soltanto per il tempo di esecuzione.

Il presente studio considera una formulazione continua della PSO (CPSO), nella quale la dinamica della particella è descritta da un'equazione differenziale ordinaria del secondo ordine e il moto a coefficienti congelati è propagato analiticamente [2]. L'aggettivo *continuo* si riferisce alla variabile temporale del modello, non a uno spazio decisionale continuo in contrapposizione a uno discreto: gli esperimenti qui considerati ottimizzano funzioni scalari su domini continui e limitati. La formulazione matematica adottata, incluse le convenzioni operative utilizzate dal software, è stata verificata e fissata nella fase di sviluppo del modello. Il problema affrontato in questo articolo è quindi sperimentale e computazionale: stabilire quali semantiche parallele preservino la traiettoria CPSO, quali introducano dinamiche differenti, come la struttura sociale influenzi la ricerca e in quali condizioni il parallelismo CPU riduca effettivamente il tempo di calcolo.

La domanda centrale non è soltanto se sia possibile distribuire il lavoro tra più thread, ma **quale parte del processo CPSO possa essere parallelizzata senza cambiare l'algoritmo**. Il primo livello riguarda gli esecutori sincroni: le particelle leggono lo stesso snapshot dei migliori personali, i calcoli indipendenti vengono eseguiti in parallelo e i risultati vengono applicati con un commit deterministico. Master–slave parallelizza le valutazioni obiettivo; particle-parallel parallelizza anche il lavoro locale delle particelle. In presenza di funzione obiettivo deterministica e thread-safe, medesimi stream casuali e ordine di commit, queste due modalità sono progettate per restituire la stessa traiettoria osservata del riferimento seriale. Lo scaling viene dunque misurato solo dopo avere verificato tale equivalenza.

Il secondo livello riguarda modalità che cambiano la dinamica. Nell'aggiornamento asincrono event-driven ogni completamento può diventare immediatamente visibile alla popolazione; non esiste una barriera globale che imponga a

tutte le particelle di osservare lo stesso stato. Nel modello a isole, invece, la popolazione è suddivisa in sottopopolazioni che evolvono localmente e si scambiano emigranti secondo una politica di migrazione. Per queste modalità il confronto viene effettuato a popolazione e budget FE totale controllati, ma non viene interpretato come misura di speedup rispetto al seriale: la qualità può cambiare proprio perché cambiano le interazioni.

La rete sociale è un ulteriore fattore sperimentale, distinto dall'esecutore. Il software implementa sette famiglie: global-best, ring periodico, griglia toroidale di von Neumann, wheel, random connessa, small-world di Watts–Strogatz e scale-free di Barabási–Albert. La campagna architetturale principale usa global, ring e von Neumann come riferimenti canonici, coerenti con la letteratura classica sulla struttura di popolazione nella PSO [3, 4]. Una campagna separata estende l'indagine alle altre quattro famiglie. L'espansione è motivata sia dalla letteratura sui grafi small-world e scale-free [5–7], sia da un'analisi recente che osserva proprietà small-world e distribuzioni dei gradi a coda pesante nelle reti di interazione ricostruite dalla dinamica PSO [8]. Le famiglie implementate hanno gradi e connettività differenti: il confronto è quindi una valutazione tra modelli canonici di grafo, non un esperimento causale che isola la sola forma della rete a grado medio fissato.

La qualità della ricerca viene valutata a due livelli. Il primo è un insieme interno di sei funzioni deterministiche (Sphere, Rastrigin, Rosenbrock, Ackley, Griewank e Salomon) in dimensione 30 e 100: consente di confrontare in modo appaiato topologie e semantiche architetturali in un ambiente controllato, ma non sostituisce benchmark standardizzati. Il secondo usa COCO BBOB, BBOB-LargeScale e CEC 2013 LSGO, per collocare la CPSO rispetto a una famiglia di PSO discrete in spazio continuo e verificare comportamenti su testbed riconosciuti [9–11]. La misura di qualità non va confusa con quella del parallelismo: i driver esterni utilizzano callback seriali e non forniscono evidenza sullo scaling del motore nativo.

Il confronto algoritmico comprende dieci baseline PSO selezionate per coprire differenti meccanismi di attrazione, informazione e organizzazione della popolazione: global-best classica, inerzia, fattore di costrizione, local-best ad anello, CLPSO, SL-PSO, SPSO-2011, FIPS, Competitive Swarm Optimizer e Dynamic Multi-Swarm PSO. La selezione è intenzionalmente interna alla famiglia PSO: non viene presentata come una comparazione esaustiva con tutte le metaeuristiche contemporanee, quali CMA-ES o le famiglie differential-evolution-based. Le ragioni della selezione e i riferimenti diretti sono discussi nella sezione successiva.

Le domande di ricerca sono:

- **RQ1 – Invarianza sincrona.** Master–slave e particle-parallel preservano posizione, valore e intera cronologia best-so-far del riferimento seriale sotto il contratto di aggiornamento implementato?
- **RQ2 – Scalabilità CPU.** In quali combinazioni di dimensione, popolazione e costo della valutazione le modalità sincrone parallele producono speedup e quale efficienza mantengono all'aumentare dei worker?
- **RQ3 – Dinamiche alternative.** A parità di FE e popolazione totale, come cambiano qualità e successo quando gli aggiornamenti sono event-driven o la popolazione è organizzata in isole con migrazione?
- **RQ4 – Topologie.** Le sette famiglie di grafo producono differenze riproducibili nella qualità CPSO e nel costo di selezione sociale? Le conclusioni sono limitate alle famiglie e ai parametri effettivamente testati.
- **RQ5 – Posizionamento algoritmico.** Come si colloca la CPSO, a budget FE abbinato, rispetto a dieci baseline PSO discrete su BBOB, BBOB-LargeScale e CEC 2013 LSGO?

Il contributo non consiste nell'introdurre per la prima volta la parallel PSO, l'aggiornamento asincrono o i modelli a isole: tali paradigmi sono ampiamente studiati. Il contributo che intendiamo valutare è una separazione esplicita fra (a) parallelismo dell'esecutore con traiettoria invariata, (b) parallelizzazione con dinamica modificata e (c) struttura della comunicazione sociale, applicata allo stesso modello CPSO e accompagnata da protocolli FE-matched, controlli di equivalenza, testbed standard e tracciamento riproducibile. La portata effettiva delle conclusioni dipenderà dai risultati delle campagne, non dalla sola ampiezza del software.

2 Stato dell'arte e posizionamento

2.1 Parallelizzazione della PSO: esecutori e semantica

La letteratura sulla parallel PSO comprende modelli master–slave, parallelismo a grana fine, aggiornamenti asincroni, accelerazione GPU e modelli a isole. Le prime analisi generali del parallelismo negli algoritmi evolutivi chiariscono la distinzione tra distribuzione della valutazione, parallelismo interno alla popolazione e decomposizione in sottopopolazioni [12, 13]; le rassegne specifiche PSO organizzano i metodi in funzione della granularità, della sincronizzazione e della piattaforma [14]. Le implementazioni parallele per problemi di ottimizzazione numerica mostrano che la valutazione indipendente delle particelle può essere sfruttata in architetture globali o master–slave [15], mentre lavori successivi hanno studiato esplicitamente gli aggiornamenti asincroni [16] e le implementazioni su GPU [17].

Tra i lavori più direttamente affini al livello esecutivo si colloca PAPSO di Li et al. [18], che combina thread pool fork–join, comunicazione non bloccante e work stealing per una PSO asincrona su CPU multicore. Gli autori confrontano il proprio framework con OpenMP e con l'esecuzione non parallela, riportando miglioramenti di tempo e scaling nel proprio ambiente sperimentale. Charilogis et al. [19] propongono invece modifiche alla PSO e meccanismi di propagazione delle particelle migliori tra unità parallele, con una valutazione in termini di chiamate alla funzione e tempo. Si tratta di lavori pertinenti, ma non costituiscono un confronto numerico diretto con il presente studio: cambiano algoritmo, parametri, hardware, funzioni e granularità delle misure.

Un ulteriore filone considera implementazioni eterogenee CPU–GPU e modelli specifici per applicazioni intensive; ad esempio, una realizzazione parallela di ALC-PSO combina OmpSs e CUDA [20]. Più recentemente, Sato et al. hanno studiato la parallelizzazione multicore di MOPSO/D attraverso un virtual global best e una partizione dello spazio degli obiettivi [21]. Questo risultato è rilevante per il tema generale del parallelismo swarm, ma il suo problema è multi-obiettivo e

il meccanismo di decomposizione non è equivalente al modello single-objective qui considerato. Analogamente, lavori 2026 su PSO multicore-aware per edge offloading e su varianti parallelizzate per training di reti neurali sono applicazioni ibride e specifiche di dominio, non baseline direttamente trasferibili al protocollo single-objective [22, 23].

La nostra distinzione fra esecutori sincroni e algoritmi con dinamica modificata è quindi complementare, non sostitutiva, a questi lavori. La valutazione non mira a dimostrare che un backend CPU sia universalmente più veloce di PAPSO, OpenMP o GPU; mira a verificare se due strategie di parallelizzazione del medesimo modello CPSO preservino il riferimento seriale e a misurare il loro scaling rispetto a quel riferimento. L'asincronia event-driven e le isole vengono invece analizzate come regimi algoritmici autonomi, non come esecutori equivalenti.

2.2 Topologie sociali e modelli a isole

La struttura del vicinato è una componente fondativa della PSO. I lavori di Kennedy e Mendes documentano il ruolo della struttura di popolazione e delle topologie di informazione nella prestazione dell'algoritmo [3, 4]. L'anello e il global-best costituiscono due estremi utili: il primo limita la diffusione dell'informazione a un vicinato locale, il secondo rende disponibile il migliore globale a tutta la popolazione. La griglia toroidale di von Neumann rappresenta una struttura locale regolare a due dimensioni. Queste tre topologie sono mantenute nel protocollo architetturale principale perché forniscono riferimenti facilmente interpretabili e consolidati.

Le quattro topologie aggiuntive ampliano lo spettro di connettività: wheel separa un hub dalle periferie; random connessa introduce irregolarità senza imporre una geometria regolare; Watts–Strogatz controlla il rewiring di un reticolo locale e quindi il compromesso tra clustering e distanze; Barabási–Albert introduce attaccamento preferenziale e nodi ad alto grado. Le ultime due richiamano proprietà di rete studiate in modo indipendente dalla PSO [5, 6]; la variante scale-free è stata già esplorata direttamente nella PSO da Liu et al. [7]. Un'analisi del 2025 costruisce reti di interazione dalle traiettorie PSO e riporta caratteristiche small-world e distribuzioni dei gradi a coda pesante su tredici funzioni benchmark [8]. Questo lavoro rafforza la motivazione di includere topologie non regolari, ma non dimostra che una delle famiglie implementate sia universalmente ottimale.

I modelli a isole sono un paradigma più generale degli algoritmi evolutivi: ciascuna isola evolve una popolazione locale e scambia individui secondo una topologia e una politica di migrazione. Da Silveira et al. analizzano topologie e politiche di migrazione in modelli sincroni e asincroni per algoritmi genetici [24]; lavori successivi introducono isole eterogenee e riconfigurabili [25]. Sono precedenti pertinenti per la progettazione della migrazione, ma non sono confronti PSO continui: l'algoritmo locale e i problemi applicativi sono diversi. Nel nostro modello le isole eseguono la stessa CPSO locale; la variabile di studio è la decomposizione della popolazione, il numero di isole, la topologia intra-isola e la politica di migrazione.

La campagna a sette topologie non è un disegno degree-matched: le famiglie hanno per costruzione distribuzioni del grado differenti. Pertanto il risultato sarà interpretato come confronto tra istanze canoniche di famiglie di grafo, non come prova causale che attribuisca la differenza osservata esclusivamente alla forma geometrica della rete.

2.3 Baseline PSO e testbed di valutazione

Le dieci baseline sono scelte per coprire assi distinti della letteratura PSO, non per massimizzare il numero dei metodi. Le quattro varianti classiche (global-best standard, inerzia lineare, fattore di costrizione e local-best ring) forniscono riferimenti facilmente riconoscibili e collegano le principali scelte di aggiornamento e topologia [1, 4, 26, 27]. CLPSO e SL-PSO rappresentano meccanismi di apprendimento sociale più articolati, con motivazioni di diversificazione e scalabilità [28, 29]. SPSO-2011 fornisce un riferimento standardizzato della famiglia PSO [30]. FIPS esplicita l'uso congiunto delle informazioni di tutti i vicini [31]. CSO rappresenta l'apprendimento competitivo per problemi large-scale [32]; DMS-PSO introduce sottosciami dinamici e riorganizzazione periodica [33]. L'insieme confronta quindi aggiornamenti globali e locali, inerzia e costrizione, apprendimento per esemplari, informazione fully-informed, competizione e multi-swarm.

Per la valutazione black-box, COCO BBOB offre 24 funzioni single-objective noiseless e protocolli basati sul tempo di raggiungimento di target di qualità; BBOB-LargeScale estende le funzioni a dimensioni maggiori, fino a 640 nella suite standard [9, 10]. L'analisi anytime e le ECDF dei runtime-to-target sono riferimenti metodologici più informativi di un singolo best finale, perché misurano il numero di FE necessario per raggiungere livelli prefissati [34]. CEC 2013 LSGO introduce quindici funzioni a 1000 variabili e un protocollo con checkpoint FE espliciti [11]. Le tre suite coprono quindi ottimizzazione continua generale, problemi ad alta dimensione e benchmark large-scale strutturati.

Il confronto qui definito rimane tuttavia PSO-vs-PSO. Non include ancora CMA-ES, differential evolution (ad esempio SHADE/L-SHADE) o ottimizzatori non-PSO selezionati dal repository COCO. Di conseguenza, le conclusioni potranno riguardare il posizionamento relativo della CPSO rispetto alle dieci implementazioni PSO specificate, non una superiorità sullo stato dell'arte complessivo degli ottimizzatori continui. Un confronto con non-PSO sarebbe un'estensione separata e richiederebbe implementazioni e protocolli altrettanto controllati.

2.4 Sintesi del posizionamento

Tabella 1: Motivazione sperimentale delle sette topologie implementate. Le prime tre sono incluse nella campagna architetturale principale; le altre quattro appartengono all'espansione dedicata.

Topologia	Proprietà strutturale	Motivazione e ruolo nel disegno
Global-best	Informazione del miglior personale globale disponibile a tutti	Riferimento a massima diffusione informativa; permette di confrontare i backend sotto una topologia canonica fortemente connessa.
Ring	Vicinato locale periodico a basso grado	Riferimento classico local-best; riduce la portata della comunicazione e rende osservabile la diffusione locale dell'informazione [4].
Von Neumann	Griglia toroidale regolare con vicini ortogonali	Topologia locale regolare bidimensionale, utile per confrontare l'anello con un vicinato locale più strutturato [3].
Wheel	Un nodo hub collegato a tutti i nodi periferici; periferie non direttamente collegate	Caso di elevata centralizzazione e dipendenza da un nodo di riferimento; estensione strutturale non inclusa nel nucleo principale.
Random connessa	Archi campionati casualmente con successiva connessione delle componenti	Introduce irregolarità nella connettività senza imporre una geometria spaziale; la riparazione della connettività è parte della definizione operativa.
Small-world	Reticolo locale con rewiring di Watts–Strogatz	Combina clustering locale e scorciatoie; motivata dalle proprietà small-world osservate in reti di interazione PSO [5, 8].
Scale-free	Attaccamento preferenziale con presenza di nodi hub	Introduce una distribuzione del grado eterogenea e code pesanti; è stata studiata anche in modelli PSO [6–8].

Tabella 2: Corrispondenza tra le dieci baseline PSO, il riferimento bibliografico e il meccanismo che ciascuna baseline porta nel confronto. Le implementazioni e gli adattamenti del protocollo sono specificati nella sezione 4.6.

Baseline	Riferimento	Ragione dell'inclusione
Classic global-best	Kennedy (1995) [1]	Riferimento storico della dinamica gbest, necessario per ancorare il confronto a una PSO canonica.
Inertia global-best	Shi e Eberhart (1998) [26]	Isola il ruolo dell'inerzia e della sua schedule nella gestione del compromesso esplorazione–sfruttamento.
Constriction global-best	Clerc (2002) [27]	Introduce una parametrizzazione basata sul fattore di costrizione, distinta dalla schedule inerziale.
Inertia local-best ring	Kennedy e Mendes (2002) [4]	Controlla l'effetto di un'informazione sociale locale rispetto al riferimento global-best.
CLPSO	Liang et al. (2006) [28]	Rappresenta l'apprendimento per esemplari coordinata-per-coordinata e una strategia di diversificazione.
SL-PSO	Cheng e Jin (2015) [29]	Porta un meccanismo di social learning specificamente proposto per problemi scalabili.
SPSO-2011	Zambrano-Bigiarini et al. (2013) [30]	Fornisce un riferimento standardizzato della famiglia PSO, pur senza rivendicare identità bitwise con una specifica implementazione esterna.
FIPS	Mendes et al. (2004) [31]	Usa contributi aggregati dei vicini invece di un singolo leader sociale.
CSO	Cheng e Jin (2015) [32]	Introduce competizione a coppie e aggiornamento dei perdenti, senza la stessa regola di attrazione al best globale.
DMS-PSO	Liang e Suganthan (2005) [33]	Rappresenta sottosciami dinamici e riorganizzazione periodica della popolazione.

Tabella 3: Mappa dei filoni bibliografici rispetto alle domande sperimentali del presente lavoro. La colonna finale chiarisce il tipo di confronto: diretto sul tema, ma non necessariamente numericamente comparabile.

Filone	Riferimenti rappresentativi	Collegamento con il presente studio
Parallel PSO CPU / asincrona	Schutte et al. (2004); Koh et al. (2006); Li et al. (2023); Charilogis et al. (2023)	Parallelismo della valutazione, thread pool, scheduling asincrono e sincronizzazione delle informazioni; confronto concettuale con i nostri backend, non confronto cronometrico tra hardware eterogenei.
CPU/GPU e PSO applicative	Mussi et al. (2011); Yazdanpanah e Alaei (2024); Verma et al. (2026)	Mostrano l'estensione a GPU, CPU eterogenee e applicazioni; il nostro perimetro attuale è CPU shared-memory e benchmark single-objective.
Topologia e network science	Kennedy (1999); Kennedy e Mendes (2002); Liu et al. (2014); Deng e Liu (2025)	Motivano le topologie locali, small-world e scale-free e l'analisi esplicita della rete di comunicazione.
Parallel island models	da Silveira et al. (2023, 2024)	Motivano topologie di migrazione e politiche di scambio; i lavori citati studiano EA/GA e non costituiscono baseline CPSO.
Benchmark black-box	Hansen et al. (2021, 2022); Varelas et al. (2020); Li et al. (2013)	Fondano la scelta di COCO BBOB/BBOB-LS e CEC 2013 LSGO e l'uso di metriche FE-to-target e checkpoint.

3 Modello matematico e semantica operativa

3.1 Stato della particella e dinamica a coefficienti congelati

Sia N il numero di particelle e D la dimensione dello spazio decisionale. Per la particella i allo step osservativo k indichiamo con $x_i^k, v_i^k \in \mathbb{R}^D$ posizione e velocità, con p_i^k la posizione del migliore personale e con s_i^k il leader sociale determinato dalla topologia scelta. Il valore obiettivo è $J(x)$. Il modello implementato aggiorna la dinamica su intervalli di durata Δt nei quali coefficienti casuali, memoria personale, informazione sociale e parametro di inerzia restano congelati.

Per ogni particella e intervallo vengono estratti due scalari $r_{1,i}^k, r_{2,i}^k \in [0, 1)$. Poniamo

$$a_i^k = c_1 r_{1,i}^k, \qquad b_i^k = c_2 r_{2,i}^k, \qquad q_i^k = a_i^k + b_i^k, \quad (1)$$

e definiamo $F_i^k = a_i^k p_i^k + b_i^k s_i^k$. Per ogni coordinata j la traiettoria locale soddisfa

$$\ddot{x}_{i,j}(t) + (1 - \mu_k)\dot{x}_{i,j}(t) + q_i^k x_{i,j}(t) = F_{i,j}^k, \quad t \in [0, \Delta t], \quad (2)$$

con condizioni iniziali $x_{i,j}(0) = x_{i,j}^k$ e $\dot{x}_{i,j}(0) = v_{i,j}^k$. Gli scalari casuali sono condivisi da tutte le coordinate della stessa particella nello stesso intervallo, secondo la convenzione del modello verificato e implementato.

Quando $q_i^k > 0$, l'equilibrio della dinamica congelata è $x_{i,j}^{\mathrm{eq}} = F_{i,j}^k / q_i^k$. La parte omogenea ha equazione caratteristica

$$\lambda^2 + (1 - \mu_k)\lambda + q_i^k = 0, \qquad \alpha_k = \frac{1 - \mu_k}{2}. \quad (3)$$

Il propagatore $\Phi_{\Delta t}$ è valutato analiticamente nei regimi sottosmorzato, criticamente smorzato e sovrasmorzato; il caso $q_i^k = 0$ è trattato separatamente. Non viene impiegato un integratore temporale numerico per approssimare la traiettoria all'interno dell'intervallo. La funzione obiettivo è valutata soltanto all'estremo $t = \Delta t$.

Per una run con K aggiornamenti, il parametro di inerzia varia secondo

$$\mu_k = \mu_{\max} - (\mu_{\max} - \mu_{\min}) \frac{k}{K - 1}, \qquad k = 0, \ldots, K - 1, \quad (4)$$

quando $K > 1$; per $K \leq 1$ viene utilizzato $\mu_{\max}$. Le posizioni iniziali sono campionate uniformemente e indipendentemente per coordinata nell'intervallo ammissibile. Le velocità iniziali seguono

$$v_{i,j}^0 \sim U\left[-\rho(u_j - \ell_j), \rho(u_j - \ell_j)\right], \quad (5)$$

con ρ pari al parametro `initial_velocity_fraction`.

3.2 Vincoli, memoria e topologia sociale

La soluzione analitica viene seguita da una mappa discreta di gestione dei vincoli. Se una coordinata oltrepassa il limite inferiore ℓ_j, la posizione viene proiettata su ℓ_j e una velocità uscente viene sostituita con $-e v_{i,j}$; il caso del limite superiore u_j è simmetrico. Il coefficiente $e \in [0, 1]$ è il parametro di restituzione. Questa mappa è una regola algoritmica applicata agli estremi degli intervalli, non una condizione al contorno dell'ODE. Non vengono rilevati attraversamenti multipli all'interno dello stesso intervallo: la posizione intermedia della traiettoria analitica non viene monitorata.

Il miglior personale è aggiornato soltanto in presenza di miglioramento stretto,

$$p_i^{k+1} = \begin{cases} x_i^{k+1}, & J(x_i^{k+1}) < J(p_i^k), \\ p_i^k, & \text{altrimenti.} \end{cases} \quad (6)$$

I pareggi nella selezione di un leader sociale sono risolti scegliendo l'indice di particella minore. Le sette topologie implementate sono: (i) globale, nella quale tutte le particelle condividono il migliore personale dell'intera popolazione; (ii) anello periodico di raggio uno; (iii) griglia toroidale di von Neumann, con sé stessa e i quattro vicini ortogonali; (iv) wheel PSO hub-and-spoke, con un solo focal particle collegato a tutte le altre particelle e senza archi tra i nodi periferici; (v) grafo random connesso ER-like, ottenuto campionando gli archi con probabilità $p = d_{\mathrm{target}}/(N - 1)$ e aggiungendo archi ponte tra componenti se necessario; (vi) small-world di

Watts–Strogatz, costruito da un reticolo ad anello di grado $k = 4$ con rewiring indipendente di probabilità $\beta = 0.15$; (vii) grafo scale-free di Barabási–Albert, generato a partire da una clique iniziale di $m + 1$ nodi con attachment preferenziale $m = 2$. Le costruzioni richiamano modelli consolidati di rete e topologia PSO [3, 5–7, 35]. Il grafo random è connesso per costruzione dopo l'aggiunta degli archi ponte; il small-world non viene riparato se il rewiring lo rende disconnesso. I grafi sono generati una volta per run da un seme dedicato e restano statici. Le topologie hanno profili di grado differenti e il confronto è quindi tra famiglie canoniche complete, non un effetto causale della sola forma del grafo a grado medio fissato. La griglia richiede $RC = N$ e dimensioni almeno pari a due; non è attualmente ammessa come topologia interna del modello a isole.

3.3 Semantica sincrona e proprietà di equivalenza

In una generazione sincrona, tutti i leader sociali vengono selezionati dallo stesso snapshot $P^k = (p_1^k, \ldots, p_N^k)$. I propagatori e le valutazioni obiettivo possono essere eseguiti in qualunque ordine perché ciascun risultato è scritto nello slot associato alla particella. Gli stati personali e globali sono poi aggiornati in ordine deterministico degli indici. Master–slave parallelizza soltanto le valutazioni obiettivo; particle-parallel parallelizza anche inizializzazione e propagazione delle particelle, mantenendo lo snapshot sociale congelato e il commit ordinato.

Proposizione 3.1 (Invarianza rispetto alla schedulazione sincrona)**.** *Si considerino una configurazione e un seme fissati. Si assumano: (i) funzione obiettivo deterministica e priva di effetti collaterali, thread-safe quando invocata in parallelo; (ii) stato pseudocasuale assegnato a ciascuna particella e consumato esclusivamente da essa; (iii) selezione dei leader dallo stesso snapshot pre-generazione; (iv) identiche mappe di propagazione, gestione dei vincoli, regole di miglioramento e tie-breaking; (v) commit deterministico in ordine di indice. Allora le esecuzioni seriale, master–slave e particle-parallel producono la stessa sequenza di stati osservati, degli incumbent e dei valori best-so-far, a meno delle differenze dovute a un diverso ambiente floating-point o a una diversa implementazione della funzione obiettivo.*

Dimostrazione. All'inizializzazione, ogni particella usa lo stream pseudocasuale associato al proprio indice e genera quindi lo stesso stato iniziale nelle tre esecuzioni. Supponiamo per induzione che, all'inizio della generazione k, posizioni, velocità, migliori personali e stati pseudocasuali coincidano. La selezione sociale dipende soltanto dallo snapshot comune P^k e dalla stessa topologia, perciò ogni particella riceve il medesimo attrattore. Lo stream privato della particella produce la stessa coppia $(r_{1,i}^k, r_{2,i}^k)$, indipendentemente dal thread che ne esegue il lavoro. La stessa mappa analitica e la stessa mappa dei vincoli producono dunque la medesima posizione candidata; la deterministica funzione obiettivo restituisce lo stesso valore. Le computazioni per particelle diverse non modificano lo stato condiviso prima del commit e i risultati vengono scritti in posizioni indicizzate. Il commit, eseguito con il medesimo ordine e le medesime regole di confronto, genera gli stessi migliori personali e lo stesso incumbent globale. Gli stream pseudocasuali hanno consumato lo stesso numero di estrazioni e restano allineati. Lo stato completo all'inizio di $k+1$ coincide, chiudendo il passo induttivo. □

La proposizione esplicita il contratto algoritmico che motiva il test di equivalenza implementativo. In aritmetica floating-point fissata e usando la stessa libreria di obiettivo, il progetto verifica l'uguaglianza esatta dei risultati osservati. Non ne segue equivalenza per funzioni obiettivo non deterministiche, non thread-safe, o per esecuzioni su compilatori e architetture floating-point differenti.

3.4 Semantica asincrona e a isole

Nell'architettura event-driven, dopo l'inizializzazione viene inserito in una coda un evento $(i, 0)$ per ciascuna particella. Un evento (i, k) legge sotto mutex il migliore personale della particella e l'attrattore sociale corrente, propaga e valuta il candidato, pubblica immediatamente un eventuale miglioramento e reinserisce $(i, k + 1)$ nella coda, se richiesto. Una particella non ha due eventi simultaneamente in volo: il successivo evento viene inserito soltanto dopo il completamento del precedente. Non esiste tuttavia una barriera globale di generazione; con più worker, l'ordine di completamento e quindi gli snapshot sociali osservati può dipendere dalla schedulazione.

Nel modello a isole la popolazione totale N è divisa in M sottopopolazioni uguali. Ogni isola esegue una generazione sincrona indipendente. Quando è previsto un evento di migrazione, gli incumbent di tutte le isole vengono copiati in uno snapshot immutabile; solo dopo aver costruito tale snapshot sono applicate le sostituzioni. La migrazione ad anello invia l'incumbent dell'isola precedente, mentre quella fully-connected seleziona il migliore incumbent tra le altre isole. L'immigrato sostituisce il peggiore migliore personale solo se strettamente migliore e la velocità del destinatario viene azzerata. Un emigrante possiede già il proprio valore obiettivo, pertanto la migrazione non aumenta il numero di valutazioni.

Tutte le modalità mantengono il budget complessivo

$$N_{\mathrm{FE}} = N(K + 1), \tag{7}$$

comprensivo della valutazione iniziale di ogni particella. Le storie sincrone sono registrate a ogni generazione; quella asincrona dopo ogni evento di aggiornamento e include esplicitamente l'asse delle valutazioni. Per costruzione, le modalità asincrone e a isole sono algoritmi con dinamiche differenti e il loro rapporto di tempo con il riferimento non viene denominato speedup parallelo.

4 Protocollo sperimentale

4.1 Disegno della campagna

La campagna predefinita è articolata in quattro stadi, separando gli effetti sulla dinamica di ricerca dalla misurazione dello scaling computazionale. Per gli stadi di qualità si considerano Sphere, Rastrigin, Rosenbrock, Ackley, Griewank e Salomon, in dimensione $D \in \{30, 100\}$, con popolazione totale $N = 64$, $K = 500$ aggiornamenti e 30 semi appaiati. Ogni run utilizza quindi $N(K + 1) = 32\,064$ valutazioni obiettivo.

Il primo stadio isola la topologia sociale in esecuzione seriale. Il secondo confronta la semantica event-driven a uno e quattro worker con i riferimenti seriali corrispondenti. Il

Tabella 4: Struttura della campagna predefinita. Gli stadi 1–3 misurano qualità con budget di valutazioni fissato; lo stadio 4 misura esclusivamente il tempo dell'esecutore su un carico sintetico.

Stadio	Confronto principale	Run	Valutazioni obiettivo
1. Topologie	Serial-global, serial-ring, serial-grid	1 080	34 629 120
2. Asincronia	Tre topologie; asincrono con 1 e 4 worker; riferimenti seriali abbinati	3 240	103 887 360
3. Isole	2, 4 e 8 isole; topologia locale e politiche di migrazione	3 960	126 973 440
4. Strong scaling	Serial, master–slave e particle-parallel; 1, 2, 4 e 8 worker	1 080	8 398 080
Totale		9 360	273 888 000

terzo studia il numero di isole, la topologia interna e le politiche di migrazione ad anello o fully-connected mantenendo costante N e il budget FE. Le configurazioni esatte e le combinazioni sono registrate in `configs/campaigns/definitive_architecture_campaign.json`.

4.2 Espansione a sette topologie

La campagna principale usa tre topologie (global-best, ring e von Neumann) per contenere il numero di combinazioni nella comparazione fra semantiche. La copertura software è più ampia: la campagna complementare `cpso_topology_expansion` aggiunge wheel, random connessa, small-world e scale-free, per un totale di sette famiglie. Il protocollo è separato dalla campagna principale e non implica la valutazione del prodotto cartesiano completo fra sette topologie e cinque modalità.

Il pilot topologico usa sei funzioni native, $D \in \{30, 100\}$, $N = 64$, 200 aggiornamenti e 10 semi; confronta le sette topologie in esecuzione seriale per 840 run. Lo stadio di costo usa `sphere_work`, $D = 30$, $N = 64$, 150 aggiornamenti e 10 semi; per ciascuna topologia confronta serial, master–slave e particle-parallel con quattro worker, per 210 run. Il pilot totale è quindi di 1 050 run. La configurazione confirmatory replica lo stesso disegno con 30 semi, per 3 150 run. Le topologie sono statiche durante ogni run e la loro costruzione è controllata da un seme dedicato. I dettagli dei parametri di generazione e le limitazioni di connettività sono quelli riportati nella sezione 3.

Nel confronto qualità i semi sono appaiati tra topologie all'interno di ogni cella funzione-dimensione. L'inferenza segue la stessa gerarchia a sei famiglie di funzione descritta sotto. Nel confronto prestazionale l'equivalenza sincrona viene controllata separatamente per ciascuna topologia prima di calcolare i rapporti di tempo. Poiché le famiglie hanno profili di grado differenti, l'esperimento confronta modelli di grafo canonici e non isola causalmente la sola geometria della rete a grado medio fissato.

4.3 Misura dello scaling

Il quarto stadio usa esclusivamente `sphere_work`, una funzione sintetica basata su Sphere con una ricorrenza aritmetica controllabile. Sono considerati $D \in \{30, 100\}$, $N \in \{64, 128\}$, $K = 80$, carichi di 0, 512 e 4 096 iterazioni per valutazione e 10 semi appaiati. Per ogni backend sincrono si misurano 1, 2, 4 e 8 worker. Il carico sintetico non è incluso nell'analisi di qualità e non viene presentato come benchmark di ottimizzazione.

Lo speedup e l'efficienza sono definiti come

$$S(p) = \frac{T_{\text{serial}}}{T_p}, \qquad E(p) = \frac{S(p)}{p}, \tag{8}$$

ma vengono riportati soltanto per master–slave e particle-parallel dopo il superamento del controllo di equivalenza esatta del paragrafo 3. Le modalità asincrona e a isole non sono confrontate in termini di speedup contro il seriale, poiché non preservano la stessa traiettoria.

4.4 Unità statistiche e inferenza

I semi sono appaiati all'interno di ciascuna coppia funzione-dimensione. Le sei funzioni costituiscono i cluster di livello superiore; le dimensioni sono misure ripetute annidate nella famiglia di funzione e i semi sono repliche appaiate annidate nella cella funzione-dimensione. Per un confronto tra metodo A e riferimento B, si calcola prima la mediana delle differenze appaiate $\log_{10}(J_A) - \log_{10}(J_B)$ tra semi in ogni cella funzione-dimensione; quindi si riducono le due dimensioni alla mediana di famiglia. Il numero di unità indipendenti per l'inferenza confermativa è pertanto sei, non dodici e non il numero complessivo dei semi.

Le differenze a livello di famiglia sono sottoposte a test signed-rank bilaterale con enumerazione esatta delle configurazioni di segno; i valori di probabilità sono corretti con la procedura di Holm all'interno della famiglia di confronti pre-specificata. L'analisi omnibus di Friedman è basata sui ranghi dei metodi all'interno di ogni famiglia e usa 20 000 permutazioni dei nomi dei metodi entro famiglia per calibrare il valore di probabilità. I valori asintotici chi-quadro e Iman–Davenport sono conservati soltanto come diagnostica secondaria. Gli intervalli al 95% sono ottenuti con bootstrap annidato: ricampionamento delle famiglie, delle dimensioni dentro ciascuna famiglia e dei semi appaiati dentro ciascuna cella.

Il numero ridotto di famiglie limita la potenza e la generalizzabilità statistica. I conteggi di vittorie, sconfitte e pareggi a livello run e le statistiche separate per funzione-dimensione sono riportati a fini descrittivi, ma non vengono trattati come osservazioni indipendenti. Il target di successo è 10^{-8} per le funzioni con ottimo globale nullo; le run senza successo sono censurate al budget FE nel calcolo dell'ERT. In assenza di successi l'ERT è infinito e deve essere accompagnato dal numero di successi.

4.5 Riproducibilità e stato dei risultati

Ogni run è identificata da funzione, dimensione, seme, metodo e hash della configurazione; il manifest conserva inoltre hash del codice, versione/ABI nativa, compilatore, flag di compilazione, CPU, sistema, versioni software e variabili d'ambiente relative al threading. I record sono salvati in JSON Lines append-only e sincronizzati su disco dopo il completamento di ciascuna run. Le campagne sono riprendibili: in fase di

Tabella 5: Protocollo predefinito per le suite esterne. I budget BBOB sono una scelta del presente studio; i parametri CEC riportati seguono il protocollo pubblicato. Le configurazioni sono predefinite ma le campagne non sono ancora state eseguite integralmente.

Suite	Problemi	Dimensioni	Istanze/semi	Budget per run
COCO BBOB	24 funzioni	$2, 3, 5, 10, 20, 40$	15 istanze	$10^4 D$ FE
COCO BBOB-LargeScale	24 funzioni	$20, 40, 80, 160, 320, 640$	15 istanze	$10^4 D$ FE
CEC 2013 LSGO	15 funzioni	$D = 1000$	25 semi	$3 \cdot 10^6$ FE

resume può essere eliminata soltanto una riga finale interrotta, mentre la corruzione di record completi determina un errore.

La campagna definitiva è definita e validata nel file di configurazione, ma non è stata eseguita integralmente nella versione corrente. I risultati dello smoke test servono soltanto a verificare orchestrazione, contabilità FE, controlli di equivalenza, analisi e generazione delle figure; non sono utilizzati per sostenere conclusioni scientifiche.

4.6 Benchmark esterni e baseline PSO discrete

La valutazione interna delle architetture non sostituisce benchmark esterni standardizzati. Per questo motivo il software include un livello distinto di valutazione della qualità della ricerca su tre famiglie di problemi: COCO `bbob`, COCO `bbob-largescale` e CEC 2013 LSGO [9–11]. I benchmark sono richiamati tramite adapter esterni e non vengono ricostruiti nel codice del progetto. Gli adapter non modificano le architetture del motore: l'obiettivo esterno è invocato attraverso un callback seriale, perché non si assume che le implementazioni Python/Cython dei benchmark siano thread-safe.

Per le due suite COCO, ogni metodo riceve un oggetto `Suite` e un `Observer` nuovi, così da separare contatori e file di log. Il driver verifica la corrispondenza tra il numero di chiamate registrate dall'oggetto COCO e il contatore FE dell'ottimizzatore; i dati observer sono mantenuti per il post-processing standard con `cocopp`. La suite BBOB standard usa le dimensioni $2, 3, 5, 10, 20, 40$, mentre BBOB-LargeScale usa $20, 40, 80, 160, 320, 640$ [10]. Per entrambe le suite il progetto seleziona un budget di $10^4 D$ valutazioni per problema e metodo. Tale budget è una scelta sperimentale del progetto e non viene presentato come requisito normativo di COCO.

Per CEC 2013 LSGO vengono utilizzate le 15 funzioni a 1000 variabili, con 25 semi indipendenti e budget massimo di $3 \cdot 10^6$ valutazioni per run. Sono registrati i valori best-so-far a $1.2 \cdot 10^5$, $6 \cdot 10^5$ e $3 \cdot 10^6$ FE, come previsto dal rapporto tecnico della competizione [11]. I limiti di ricerca sono letti dai metadati dell'adapter; la mappa di vincolo comune del presente studio applica il clipping alla frontiera e la restituzione della velocità con coefficiente 0.5. Questa è una scelta operativa uniforme dei metodi, non una proprietà intrinseca delle funzioni CEC.

I metodi confrontati sono la CPSO continua nativa e dieci baseline PSO a tempo discreto in spazio continuo implementate separatamente in NumPy. La prima baseline è una PSO global-best classica, con $c_1 = c_2 = 2$ e velocità inerziale non pesata [1]. La seconda utilizza un coefficiente di inerzia lineare da 0.9 a 0.4, con $c_1 = c_2 = 2.05$ [26]. La terza applica il fattore di costrizione

$$\chi = \frac{2}{\varphi - 2 + \sqrt{\varphi^2 - 4\varphi}}, \qquad \varphi = c_1 + c_2 > 4, \tag{9}$$

con $c_1 = c_2 = 2.05$ [27]. La quarta utilizza la stessa inerzia lineare della seconda ma sostituisce l'informazione globale con il miglior personale in un vicinato ad anello periodico di raggio uno [4]. Le prime tre baseline usano l'informazione global-best. La quinta baseline è il Comprehensive Learning PSO (CLPSO) [28]: ogni coordinata apprende dal personal best di un esemplare selezionato mediante torneo, con probabilità specifica della particella $P_{c,i} = 0.05 + 0.45[\exp(10i/(N-1)) - 1]/[\exp(10) - 1]$, $i = 0, \ldots, N-1$. L'aggiornamento utilizza $c = 1.49445$, inerzia lineare da 0.9 a 0.4, limite di velocità pari a 0.2 della larghezza del dominio e rinnovo degli esemplari dopo sette generazioni senza miglioramento del personal best. Il ciclo CLPSO è implementato per generazioni con uno snapshot congelato di posizioni e personal best; ciò evita che l'ordine di valutazione induca una dinamica asincrona non dichiarata. Per garantire un trattamento comune dei vincoli viene adottata la medesima mappa clamp–restitution usata dagli altri metodi, scelta che viene dichiarata come convenzione del presente protocollo.

La sesta baseline è SL-PSO (Social Learning PSO) [29], concepita per problemi scalabili. Il gruppo viene ordinato dal peggiore al migliore; ogni particella, eccetto la migliore, può apprendere da dimostratori con fitness migliore. Con indice di rango $i = 0, \ldots, N-2$, dimensione D, $M = 100$, $\alpha = 0.5$ e $\beta = 0.01$, la probabilità di apprendimento e il fattore di influenza sociale sono

$$P_i = \left(1 - \frac{i}{N}\right)^{\alpha \log(\lceil D/M \rceil)}, \qquad \gamma = \beta \frac{D}{M}. \tag{10}$$

Quando l'apprendimento è attivato, ogni coordinata sceglie indipendentemente un dimostratore migliore e aggiorna lo spostamento secondo

$$\Delta x_{i,j}^{t+1} = r_1 \Delta x_{i,j}^t + r_2 (x_{k(i,j),j}^t - x_{i,j}^t) + r_3 \gamma (\bar{x}_j^t - x_{i,j}^t), \tag{11}$$

con $r_1, r_2, r_3 \sim U(0,1)$ e $\bar{x}^t$ media della popolazione. La popolazione è mantenuta fissa per garantire la parità FE con gli altri metodi; ciò differisce dalla raccomandazione originale $M + \lceil D/10 \rceil$ e viene registrato come adattamento sperimentale. Anche SL-PSO utilizza la mappa comune di gestione dei vincoli.

La settima baseline è SPSO-2011 [30]. Utilizza un vicinato informant diretto generato casualmente: ogni particella informa $K = 3$ particelle selezionate casualmente e sé stessa; il grafo viene rigenerato dopo una generazione senza miglioramento del best globale. Per la particella i, indicando con p_i il personal best e con l_i il migliore personal best nel suo vicinato, il centro geometrico è

$$G_i = x_i + c \frac{p_i + l_i - 2x_i}{3}, \qquad c = \frac{1}{2} + \log 2, \tag{12}$$

con il centro a due punti $G_i = x_i + c(p_i - x_i)/2$ quando $l_i = p_i$. La nuova posizione è campionata nella ipersfera centrata in G_i e di raggio $\|G_i - x_i\|$, con direzione uniforme e raggio uniforme; si usa $\omega = (2\log 2)^{-1}$. Le coordinate sono normalizzate nel box unitario per l'aggiornamento geometrico e riportate nel dominio originale.

L'ottava baseline è FIPS (Fully Informed Particle Swarm) [31]. Invece di usare il solo miglior vicino, accumula i contributi dei due vicini in un anello periodico:

$$v_i^{t+1} = \chi v_i^t + \frac{\chi}{|N_i|} \sum_{j \in N_i} R_{ij} \odot (p_j - x_i^t), \qquad R_{ij,k} \sim U(0, \phi), \qquad \phi = 4.1, \tag{13}$$

dove $\chi = 2/(\phi - 2 + \sqrt{\phi^2 - 4\phi})$ è il fattore di costrizione. Il vicinato è fissato per tutta la run e non include la particella centrale.

La nona baseline è il Competitive Swarm Optimizer (CSO) [32]. A ogni generazione le particelle sono accoppiate casualmente; in ogni coppia il vincitore resta invariato e il perdente aggiorna velocità e posizione in funzione del vincitore, della propria velocità e della media precedente della popolazione:

$$\begin{aligned} v_l^{t+1} &= r_1 \odot v_l^t + r_2 \odot (x_w^t - x_l^t) \\ &\quad + \varphi r_3 \odot (\bar{x}^t - x_l^t), \\ x_l^{t+1} &= x_l^t + v_l^{t+1}. \end{aligned} \tag{14}$$

con $r_1, r_2, r_3 \sim U(0,1)$ e $\varphi = 0.1$. Solo i perdenti vengono rivalutati: con N pari, una generazione completa consuma $N/2$ FE. La storia è pertanto indicizzata per valutazioni effettive e non per generazioni.

La decima baseline è il Dynamic Multi-Swarm PSO (DMS-PSO) [33]. La popolazione viene partizionata in sottosciami di dimensione obiettivo $m = 3$; ogni particella è attratta dal proprio personal best e dal miglior personal best del sottosciame corrente:

$$\begin{aligned} v_i^{t+1} &= \omega_t v_i^t + 2r_1 \odot (p_i - x_i^t) \\ &\quad + 2r_2 \odot (l_i - x_i^t), \\ x_i^{t+1} &= x_i^t + v_i^{t+1}. \end{aligned} \tag{15}$$

L'inerzia decresce linearmente da 0.9 a 0.2 e i sottosciami vengono ricomposti casualmente ogni $R = 5$ generazioni. È implementata la variante base, senza l'estensione opzionale di ricerca locale quasi-Newton; l'eventuale resto della partizione è assorbito nell'ultimo gruppo, secondo la politica registrata nei metadati.

Tutti gli undici metodi condividono dimensione della popolazione, dominio, identificatore del problema e budget FE esatto. I parametri specifici, le politiche di inizializzazione e le convenzioni di gestione dei vincoli sono registrati nei metadati di ogni run; tali differenze sono parte delle formulazioni e non vengono nascoste dietro una falsa equivalenza dinamica. Le campagne confermative utilizzano $N = 40$ e budget multipli di N; per CSO ciò corrisponde a un numero intero di generazioni da $N/2$ valutazioni. I semi sono determinati da una funzione hash stabile dell'identificatore del problema e del seme di campagna. Lo stesso identificatore è associato a ogni metodo per consentire l'appaiamento statistico, ma non implica coordinate iniziali identiche: CPSO C++ e PSO NumPy utilizzano generatori casuali differenti. SPSO-2011 è allineato alla descrizione algoritmica standard qui specificata, ma non viene dichiarata identità bitwise con una specifica implementazione software esterna. FIPS è riferito alla variante ad anello a due vicini; CSO e DMS-PSO adottano la mappa di vincolo dichiarata nel protocollo. Le dieci varianti costituiscono un insieme rappresentativo e non esaustivo dello stato dell'arte.

Per l'esecuzione locale di sviluppo sono già predisposti tre profili ridotti, che mantengono identica la popolazione e il budget FE fra gli undici metodi della singola suite. Il pilot BBOB seleziona sette funzioni, $D \in \{5, 20\}$, cinque istanze e budget $10^3 D$, per 770 run. Il profilo BBOB-LargeScale preconfirmatory seleziona otto funzioni, $D \in \{20, 80, 320, 640\}$, tre istanze e budget $250D$, per 1 056 run. Il pilot CEC seleziona sei funzioni rappresentative, tre semi e budget massimo 120 000 FE, con checkpoint a 40 000, 80 000 e 120 000 FE, per 198 run. Questi profili sono destinati a validare il comportamento su più famiglie e a individuare problemi di integrazione o configurazione; i budget ridotti e la copertura parziale non consentono di sostituire i protocolli confirmatory.

Il protocollo CEC completo usa tutte le 15 funzioni, 25 semi e $3 \cdot 10^6$ FE per run. I sei problemi del pilot non sono campionati per ottenere una media rappresentativa dell'intera suite; servono a coprire categorie differenti del testbed in una fase preliminare. Analogamente, il pilot BBOB e il profilo BBOB-LargeScale non devono essere aggregati a campagne con budget o istanze differenti.

Con undici metodi, le campagne confirmatory complete comprendono 23 760 run metodo-problema per ciascuna suite COCO e 4 125 run per CEC 2013 LSGO. Il carico totale pianificato è pari a 3.168 miliardi di FE per BBOB, 49.896 miliardi per BBOB-LargeScale e 12.375 miliardi per CEC 2013 LSGO. Gli smoke test verificano solo integrazione, budget FE, serializzazione e resume; non sono utilizzati per classificare gli algoritmi. La valutazione COCO usa il post-processing ufficiale Observer/cocopp e le metriche runtime-to-target/ECDF; le misure di tempo reale raccolte dal driver sono riportate separatamente e non sostituiscono le FE-to-target. Nel presente manoscritto non sono ancora riportati risultati ottenuti con le suite esterne.

5 Risultati sperimentali

Questa sezione sarà compilata dopo l'esecuzione delle campagne. I pilot locali servono a verificare le pipeline e a formulare ipotesi; le conclusioni confermative saranno basate sui risultati ottenuti con i protocolli e le configurazioni congelati. Ogni tabella deve riportare il numero di run complete, le FE effettive, i seed e le funzioni incluse; i risultati non completi non devono essere aggregati come se fossero una campagna conclusa.

5.1 Equivalenza delle modalità sincrone

Da riportare: percentuale di run appaiate con identità esatta di best value, posizione e cronologia best-so-far; eventuali mismatch, loro riproducibilità e causa. L'esito deve essere verificato per master–slave e particle-parallel a ciascun numero di worker. Non è sufficiente confrontare soltanto il valore finale.

Da generare: `fig_01_synchronous_equivalence.pdf`.

5.2 Scaling e costo dell'esecutore

Da riportare: tempi appaiati, speedup, efficienza parallela e intervalli di incertezza per master–slave e particle-parallel.

Separare i tre livelli di costo della funzione obiettivo (nessun lavoro aggiuntivo, carico intermedio e carico elevato) e le dimensioni/population size previste. I tempi assoluti sono riferiti esclusivamente alla macchina descritta nel manifest.

Da generare: `fig_02_strong_scaling.pdf`.

5.3 Aggiornamento asincrono

Da riportare: qualità best-so-far e successo a budget FE fissato, distribuzione dei tempi, sensibilità al numero di worker e alla topologia sociale. Il confronto con il seriale è algoritmico, non uno speedup a traiettoria invariata.

Da generare: `fig_05_asynchronous_quality.pdf`.

5.4 Architettura a isole

Da riportare: confronto FE-matched al variare di numero di isole, topologia intra-isola e politica di migrazione; quota di migrazioni accettate e costi di comunicazione, se disponibili. Distinguere l'effetto della decomposizione da quello della migrazione.

Da generare: `fig_06_island_quality.pdf`.

5.5 Espansione alle sette topologie

Da riportare: qualità per funzione e dimensione, ranghi aggregati con unità statistiche corrette, costo della selezione sociale e verifica dell'equivalenza tra esecutori sincroni per topologia. La campagna a sette topologie non è confusa con la campagna architetturale principale a tre topologie.

Da generare: `fig_08_current_topology_quality.pdf`.

5.6 Benchmark esterni: COCO BBOB e BBOB-LargeScale

Per ogni suite saranno riportate le ECDF e le misure runtime-to-target prodotte con il post-processing ufficiale COCO. Le tabelle accessorie con i ranghi finali sono descrittive e non sostituiscono le metriche anytime. I risultati BBOB standard e BBOB-LargeScale saranno analizzati separatamente.

Da generare: report COCO/cocopp e ranghi descrittivi BBOB.

5.7 Benchmark esterno: CEC 2013 LSGO

Da riportare: andamento best-so-far ai checkpoint FE, statistiche per funzione, mediane aggregate e confronti appaiati. I risultati del pilot a sei funzioni, tre semi e 120 000 FE saranno esplicitamente descritti come preliminari; le conclusioni sulla suite richiederanno tutte le funzioni, i seed e il budget confirmatory.

Da generare: riepilogo CEC ai checkpoint FE.

5.8 Sintesi dei risultati

Questa sottosezione conterrà una sintesi delle risposte alle RQ1–RQ5, separando invarianti dimostrate e verificate, osservazioni empiriche e ipotesi non supportate. Non saranno formulate graduatorie universali sulla base dei soli pilot locali.

6 Discussione

La discussione sarà organizzata intorno alla distinzione fra equivalenza semantica e prestazione dell'esecutore. Per le modalità sincrone, ogni accelerazione dovrà essere interpretata alla luce del controllo di traiettoria esatta e del costo della funzione obiettivo. Per asincronia e isole, le variazioni della qualità saranno discusse come effetti di una diversa dinamica di aggiornamento e comunicazione, non come accelerazioni del medesimo algoritmo.

6.1 Implicazioni per la parallelizzazione CPSO

Da completare con il rapporto fra granularità del lavoro, overhead di scheduling, sincronizzazione, dimensione della popolazione e costo della valutazione. Il confronto con PAPSO, le implementazioni CPU/GPU e i recenti lavori multicore dovrà essere qualitativo e contestualizzato, senza confrontare tempi assoluti ottenuti su hardware, algoritmi o protocolli differenti.

6.2 Ruolo delle topologie e della migrazione

Da completare distinguendo qualità, densità/connettività, grado medio, costo della selezione dei vicini e frequenza delle comunicazioni. In assenza di un disegno degree-matched, non attribuire causalmente le differenze alla sola geometria del grafo.

6.3 Confronto con le baseline e validità esterna

Da completare discutendo gli esiti per famiglia di benchmark e meccanismo PSO. Il confronto rimane limitato alle dieci baseline PSO implementate e non permette conclusioni di superiorità rispetto a CMA-ES, differential evolution o all'intero stato dell'arte dell'ottimizzazione continua.

6.4 Minacce alla validità e limiti

Da documentare: numero di funzioni e unità statistiche; hardware locale fanless e variabilità termica; differenze di generatore casuale e inizializzazione fra implementazioni; adattamenti delle baseline per FE parity; callback Python seriali nei benchmark esterni; differenze fra pilot locale e campagne HPC; generalizzazione a obiettivi costosi, GPU e sistemi distribuiti.

7 Conclusioni

Da completare esclusivamente dopo le campagne confirmatory. Le conclusioni risponderanno alle cinque domande di ricerca distinguendo: (i) proprietà semantiche e di equivalenza degli esecutori sincroni; (ii) scaling misurato sulla piattaforma specificata; (iii) effetti empirici dell'asincronia e delle isole; (iv) risultati sulle sette famiglie topologiche; (v) posizionamento rispetto alle dieci baseline PSO sui benchmark esterni. Saranno riportati soltanto risultati sostenuti dai dati e saranno esplicitati i limiti di hardware, copertura e generalizzazione.

Disponibilità di dati e codice

Il codice sorgente e le configurazioni sperimentali sono inclusi nella repository di sviluppo associata al manoscritto. Le

configurazioni confirmatory sono definite nella repository; alla data di questa versione non sono ancora state eseguite integralmente. I pilot locali saranno etichettati come esplorativi e non sostituiranno i risultati confirmatory. I dati grezzi, i manifest e le figure verranno associati alla revisione del manoscritto che li discuterà.

Riferimenti bibliografici

[1] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 – International Conference on Neural Networks*, volume 4, pages 1942–1948, Perth, Australia, 1995. IEEE. doi: 10.1109/ICNN.1995.488968.

[2] Calogero Orlando and Angela Ricciardello. Analytic solution of the continuous particle swarm optimization problem. *Optimization Letters*, 15(6):2005–2015, 2021. doi: 10.1007/s11590-020-01671-3.

[3] James Kennedy. Small worlds and mega-minds: Effects of neighborhood topology on particle swarm performance. In *Proceedings of the 1999 Congress on Evolutionary Computation*, volume 3, pages 1931–1938. IEEE, 1999. doi: 10.1109/CEC.1999.785509.

[4] James Kennedy and Rui Mendes. Population structure and particle swarm performance. In *Proceedings of the 2002 Congress on Evolutionary Computation (CEC'02)*, volume 2, pages 1671–1676, Honolulu, HI, USA, 2002. IEEE. doi: 10.1109/CEC.2002.1004493.

[5] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, 1998. doi: 10.1038/30918.

[6] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999. doi: 10.1126/science.286.5439.509.

[7] Chen Liu, Wen-Bo Du, and Wen-Xu Wang. Particle swarm optimization with scale-free interactions. *PLOS ONE*, 9(5):e97822, 2014. doi: 10.1371/journal.pone.0097822.

[8] Lingyun Deng and Sanyang Liu. Collective dynamics of particle swarm optimization: A network science perspective. *Physica A: Statistical Mechanics and its Applications*, 675:130778, 2025. doi: 10.1016/j.physa.2025.130778.

[9] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1):114–144, 2021. doi: 10.1080/10556788.2020.1808977.

[10] Konstantinos Varelas, Ouassim Ait Elhara, Dimo Brockhoff, Nikolaus Hansen, Duc Manh Nguyen, Tea Tušar, and Anne Auger. Benchmarking large-scale continuous optimizers: the bbob-largescale testbed, a COCO software guide and beyond. *Applied Soft Computing*, 97:106737, 2020. doi: 10.1016/j.asoc.2020.106737.

[11] Xiaodong Li, Ke Tang, Mohammad N. Omidvar, Zhenyu Yang, and Kai Qin. Benchmark functions for the CEC'2013 special session and competition on large-scale global optimization. Technical report, Evolutionary Computation and Machine Learning Group, RMIT University, Melbourne, Australia, 2013.

[12] Enrique Alba and Marco Tomassini. Parallelism and evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 6(5):443–462, 2002. doi: 10.1109/TEVC.2002.800880.

[13] Erick Cantú-Paz. *Efficient and Accurate Parallel Genetic Algorithms*, volume 1 of *Genetic Algorithms and Evolutionary Computation*. Kluwer Academic Publishers, 2001. ISBN 978-0-7923-7221-9. doi: 10.1007/978-1-4615-4369-5.

[14] Soniya Lalwani, Harish Sharma, Suresh C. Satapathy, Kusum Deep, and Jagdish C. Bansal. A survey on parallel particle swarm optimization algorithms. *Arabian Journal for Science and Engineering*, 44(4):2899–2923, 2019. doi: 10.1007/s13369-018-03713-6.

[15] Jaco F. Schutte, Jeffrey A. Reinbolt, Benjamin J. Fregly, Raphael T. Haftka, and Alan D. George. Parallel global optimization with the particle swarm algorithm. *International Journal for Numerical Methods in Engineering*, 61(13):2296–2315, 2004. doi: 10.1002/nme.1149.

[16] Byung-Il Koh, Alan D. George, Raphael T. Haftka, and Benjamin J. Fregly. Parallel asynchronous particle swarm optimization. *International Journal for Numerical Methods in Engineering*, 67:578–595, 2006. doi: 10.1002/nme.1646.

[17] Luca Mussi, Fabio Daolio, and Stefano Cagnoni. Evaluation of parallel particle swarm optimization algorithms within the CUDA architecture. *Information Sciences*, 181(20):4642–4657, 2011. doi: 10.1016/j.ins.2010.08.045.

[18] Ming Li, Linhao Huang, Gangyan Xu, and Kong Biao. A parallel particle swarm optimization framework based on a fork-join thread pool using a work-stealing mechanism. *Applied Soft Computing*, 145:110537, 2023. doi: 10.1016/j.asoc.2023.110537.

[19] Vasileios Charilogis, Ioannis G. Tsoulos, and Alexandros Tzallas. An improved parallel particle swarm optimization. *SN Computer Science*, 4:766, 2023. doi: 10.1007/s42979-023-02227-9.

[20] Fahimeh Yazdanpanah and Mohammad Alaei. An approach for low-power heterogeneous parallel implementation of ALC-PSO algorithm using OmpSs and CUDA. *Parallel Computing*, 120:103084, 2024. doi: 10.1016/j.parco.2024.103084.

[21] Mikiko Sato, Mitsuhiko Uchida, and Yuji Sato. Parallelization of MOPSO/D on multi-core CPUs with virtual global best and overlap-aware partitioning. *Artificial Life and Robotics*, 31:536–546, 2026. doi: 10.1007/s10015-025-01093-3. Published online 20 December 2025.

[22] Vandna Rani Verma, Pushkar, Bablu Kumar, Anshul Verma, and Vishnu Sharma. Multicore aware Lyapunov based particle swarm optimization framework for efficient edge task offloading. *Discover Computing*, 29:334, 2026. doi: 10.1007/s10791-026-10224-8.

[23] Konstantinos G. Barkas, Ioannis G. Tsoulos, Vasileios Charilogis, and Glykeria Kyrou. Parallel optimized particle swarm optimization: An efficient parallel approach for neural network training. *Mathematics*, 14(19):3599, 2026. doi: 10.3390/math14193599.

[24] Lucas A. da Silveira, Thaynara A. de Lima, Jessé B. de Barros, José L. Soncco-Álvarez, Carlos H. Llanos, and Mauricio Ayala-Rincón. On the behavior of parallel island models. *Applied Soft Computing*, 148:110880, 2023. doi: 10.1016/j.asoc.2023.110880.

[25] Lucas A. da Silveira, Thaynara A. de Lima, and Mauricio Ayala-Rincón. On reconfiguring heterogeneous parallel island models. *Swarm and Evolutionary Computation*, 89:101624, 2024. doi: 10.1016/j.swevo.2024.101624.

[26] Yuhui Shi and Russell C. Eberhart. A modified particle swarm optimizer. In *Proceedings of the 1998 IEEE International Conference on Evolutionary Computation*, pages 69–73, Anchorage, AK, USA, 1998. IEEE. doi: 10.1109/ICEC.1998.699146.

[27] Maurice Clerc and James Kennedy. The particle swarm – explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1):58–73, 2002. doi: 10.1109/4235.985692.

[28] Jing J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar. Comprehensive learning particle swarm optimizer for global optimization of multimodal functions. *IEEE Transactions on Evolutionary Computation*, 10(3):281–295, 2006. doi: 10.1109/TEVC.2005.857610.

[29] Ran Cheng and Yaochu Jin. A social learning particle swarm optimization algorithm for scalable optimization. *Information Sciences*, 291:43–60, 2015. doi: 10.1016/j.ins.2014.08.039.

[30] Mauricio Zambrano-Bigiarini, Maurice Clerc, and Raúl Rojas. Standard particle swarm optimisation 2011 at CEC-2013: A baseline for future PSO improvements. In *Proceedings of the 2013 IEEE Congress on Evolutionary Computation*, pages 2337–2344. IEEE, 2013. doi: 10.1109/CEC.2013.6557848.

[31] Rui Mendes, James Kennedy, and José Neves. The fully informed particle swarm: Simpler, maybe better. *IEEE Transactions on Evolutionary Computation*, 8(3):204–210, 2004. doi: 10.1109/TEVC.2004.826074.

[32] Ran Cheng and Yaochu Jin. A competitive swarm optimizer for large scale optimization. *IEEE Transactions on Cybernetics*, 45(2):191–204, 2015. doi: 10.1109/TCYB.2014.2322602.

[33] Jing J. Liang and P. N. Suganthan. Dynamic multi-swarm particle swarm optimizer. In *Proceedings of the 2005 IEEE Swarm Intelligence Symposium*, pages 124–129. IEEE, 2005. doi: 10.1109/SIS.2005.1501611.

[34] Nikolaus Hansen, Anne Auger, Dimo Brockhoff, and Tea Tušar. Anytime performance assessment in black-box optimization benchmarking. *IEEE Transactions on Evolutionary Computation*, 26(6):1293–1305, 2022. doi: 10.1109/TEVC.2022.3210897.

[35] Paul Erdős and Alfréd Rényi. On random graphs. i. *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.